



[Список тренингов](#) / [Промпт-инжиниринг](#) / [Промпт-инжиниринг 2.0](#) / [Модуль 9. Оптимизация затрат на ИИ. Стратегии развертывания ИИ-моделей \(/teach/control/stream/view/id/931478150\)](#)

РЕ9.4 Российское облачное решение Yandex Cloud

Предыдущий урок

[\(/pl/teach/control/lesson/view?id=338667562\)](/pl/teach/control/lesson/view?id=338667562)

Следующий урок

[\(/pl/teach/control/lesson/view?id=341172968\)](/pl/teach/control/lesson/view?id=341172968)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Уважаемые студенты!

В рамках данного урока могут рассматриваться примеры передачи персональных данных при регистрации на иностранных сервисах, а также обработки и хранения пользовательских данных.

В соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных», обращаем ваше внимание: в демонстрационных примерах и при выполнении домашних заданий необходимо использовать исключительно тестовые

данные или тестовые аккаунты, в которых не указаны персональные данные — используйте вымышленные ники, дополнительные данные и т.п.

Использование иностранных сервисов не является обязательным. Вы вправе выполнять домашние задания на аналогичных разрешённых платформах по вашему выбору.

Результат дня

Узнаем как работать с облачными технологиями Яндекс, их основные функции, а также, как они могут быть полезны в работе промпт-инженера.

 **Особенности работы с Российскими нейросетями гражданами других стран (https://university.zerocoder.ru/work_ru_ai)**

Эксперт

Иван Родин:

- Постдок исследователь, Университет Катании;
- компьютерное зрение, анализ длинных видео, FPV, Action Anticipation;
- исследовательские проекты с Intel, Philips;
- публикации на CVPR, ICASSP, NAACL, CVIU

LinkedIn: <https://www.linkedin.com/in/mr-ivan-rodin/>

(<https://www.linkedin.com/in/mr-ivan-rodin/>)

TG-Channel: https://t.me/aimazing_materials
([http://https://t.me/aimazing_materials](https://t.me/aimazing_materials))

Время изучения материалов

 Время чтения и просмотра: ~ 1 час (также потребуется время для обработки данных)

 Время выполнения задания: ~ 40 мин

Инструменты урока

Yandex.Cloud (<https://yandex.cloud/ru/>) (аналоги Amazon Web Services (<https://aws.amazon.com/ru/ai/machine-learning/>), Microsoft Azure (<http://azure.microsoft.com/>), Google Cloud Platform (<https://cloud.google.com/>))

Технологии в деле

Создавайте и совершенствуйте ваши цифровые продукты с надёжной инфраструктурой и уникальными сервисами Yandex Cloud.

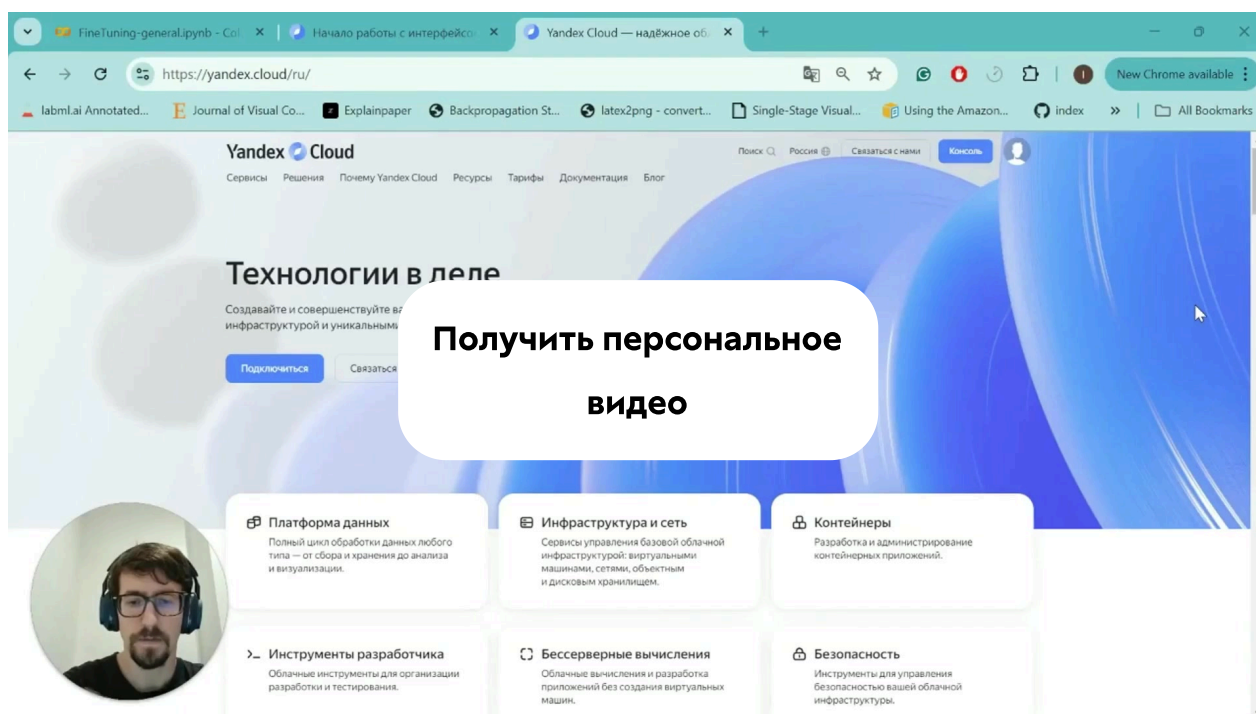
[Попробовать бесплатно](#)[Связаться с нами](#)

Облачная платформа от компании «Яндекс», предоставляющая набор инструментов для разработки, хранения и обработки данных.

Теория на сегодня

- Работа с Yandex.Cloud
- Плюсы и минусы среды разработки DataSphere
- Загрузка файлов в Yandex Object Storage
- Настройка виртуальных машин и взаимодействие с ними через командную строку

1. Работа с Yandex.Cloud



На прошлом уроке мы изучали работу с Google Cloud, а сегодня рассмотрим использование облачных технологий Яндекс.

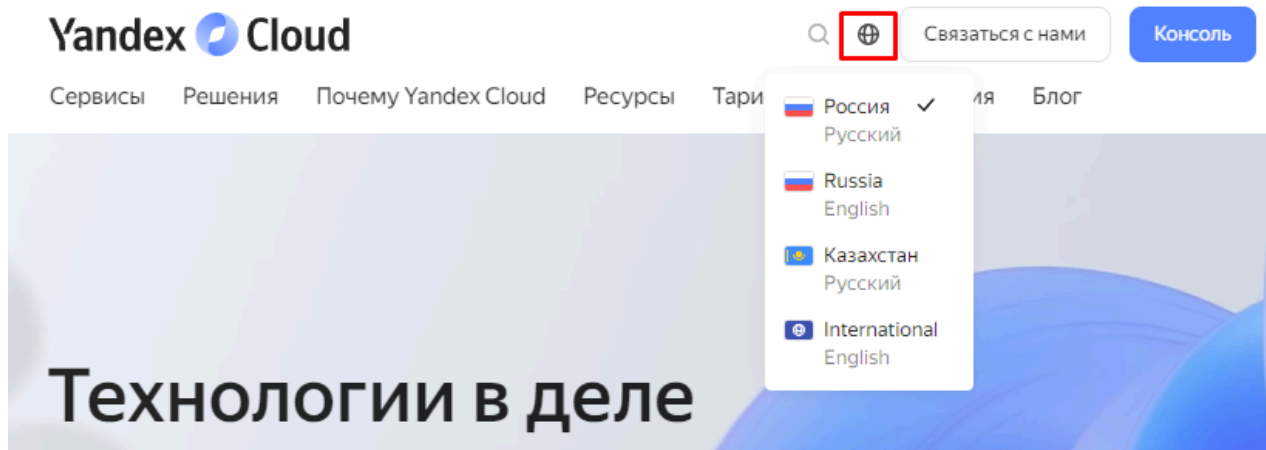
Зачем может понадобиться Яндекс.Облако?

Необходимость разработки приложения или использования значительных ресурсов для обучения нейросетевой модели с Fine-Tuning может потребовать выполнения этих задач именно на российских серверах. Это может быть обусловлено рядом причин:

- требования компании по хранению данных исключительно на территории России;
- упрощение оплаты вычислительных ресурсов российской картой, что позволяет избежать сложностей с зарубежными платформами, такими как Google Cloud, AWS или Azure.

Чтобы начать работу:

1. Переходим на сайт <https://yandex.cloud> (<https://yandex.cloud>).
2. Выбираем удобный интерфейс — на русском или английском языке.



3. Заходим в консоль, которая станет основной панелью управления всеми ресурсами и вычислениями.



При первом входе необходимо привязать банковскую карту и настроить платежный аккаунт.

Для этого:

1. Нажимаем кнопку **Создать аккаунт**.
2. Проходим регистрацию: заполняем общую информацию, персональные данные, платежные данные и контактную информацию. При привязке карты будет временно списано 11 рублей, которые затем вернутся.

💡 Рекомендуем не привязывать свою основную карту к ресурсу.

3. Нажимаем кнопку **Создать**.

Яндекс предоставляет новым пользователям **4000 рублей для ознакомления с сервисом**. Стартовый грант действует 60 дней. Этого хватит для работы на сегодняшнем уроке. В дальнейшем можно будет отслеживать траты на различные сервисы и пополнять баланс.

Создайте платёжный аккаунт и получите грант 4000 рублей на тестирование.

Стартовый грант доступен всем новым пользователям. Для его активации создайте новый платёжный аккаунт или привяжите текущий.

Создать аккаунт

Подробнее про грант

0255 0651 5173 150

💡 Важно! Для того, чтобы **избежать трат, превышающих грант**, дадим несколько простых рекомендаций:

- В Yandex Cloud можно настроить **бюджеты и лимиты на расходы**. Это позволит отслеживать затраты в режиме реального времени и получать уведомления при достижении определенного порога.

Платёжные аккаунты / account-934 / Бюджеты / Создание бюджета

Общая информация

Имя*	Zerocoder
Тип* ?	Стоимость потребления ▾
Сумма*	100 ₹
Период расчета*	Месячный ▾
Дата окончания* ?	01.2029 ✕
Уведомить* ?	Ксения Зайцева 1 ▾

Область действия

Область действия ?	Все Облака и каталоги
Сервисы	Все Выбрать

Пороги

Порог 100 ₹

...

^

Значение*	100 ₹	В процентах В валюте
Уведомить	Ксения Зайцева 1 ▾	

Добавить порог

- Отключаем или удаляем неиспользуемые ресурсы.** Например, виртуальные машины (VM), базы данных и балансировщики нагрузки могут

продолжать начислять стоимость даже в неактивном состоянии. Особое внимание уделяем тому, чтобы **виртуальная машина была удалена или отключена**.

- Регулярно проверяем раздел **«Биллинг»** для анализа ресурсов, потребляющих наибольшее количество затрат. Это помогает выявить ненужные или неэффективные расходы.
- При прохождении урока в течение нескольких дней проверяем, что все ресурсы отключены и не происходит нежелательных списаний с аккаунта.
- По завершении урока удаляем все созданные проекты (они будут отображены на главной странице).

Отнесись к этому вопросу с особым вниманием и при возникновении проблем обращайся в поддержку Яндекс.

Здесь представлена подробная информация — <https://yandex.cloud/ru/docs/getting-started/> (<https://yandex.cloud/ru/docs/getting-started/>).

Переходим в раздел **Платежный аккаунт**, где можно просмотреть гранты и расходы на вычисления.

Платежный аккаунт		Баланс	Статус	Потребление ⓘ
	account-934	0,00 ₽	Active	0,00 ₽

Платёжные аккаунты / account-934

Active

Банковская карта



Visa **** * 7119
01/2030

Изменить банковскую карту

Гранты ?

Показать все >

1 000 ₽ / 1 000 ₽

С 16.10.2024 по 15.12.2024

60 дней

Compute Cloud +2

3 000 ₽ / 3 000 ₽

С 16.10.2024 по 15.12.2024

60 дней

Все сервисы, кроме 3

Активировать промокод

На стартовой странице можно создавать проекты. Существует возможность использовать дефолтные проекты или кастомизировать их по своим нуждам, например, привязывая разные платёжные аккаунты к разным проектам. Внутри проекта создаются каталоги, которые представляют собой наборы сервисов.

Поиск



organization-kv-zayceva



cloud-kv-zayceva



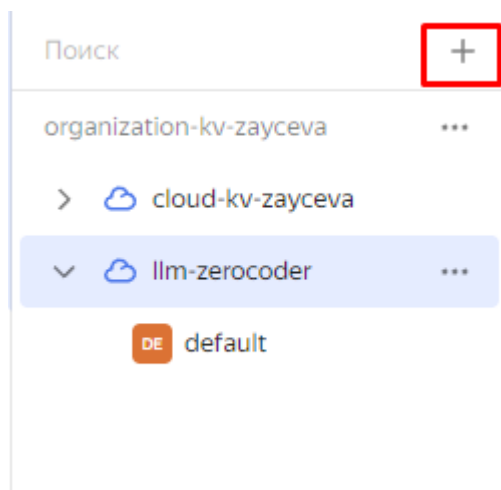
llm-zeroCoder



default

Чтобы добавить новый проект:

1. Нажимаем на плюс в правом верхнем углу панели слева.



2. Вводим имя проекта.

3. Нажимаем на кнопку **Создать**.

Внутри проекта можно создавать каталоги — наборы сервисов.
Доступно множество сервисов.

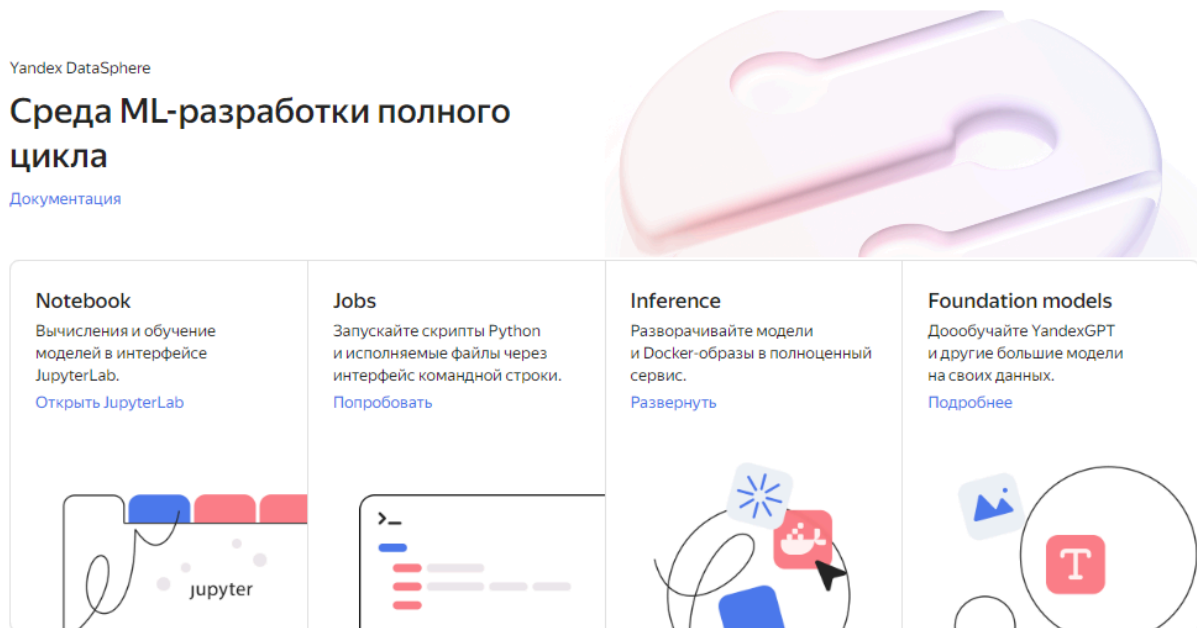
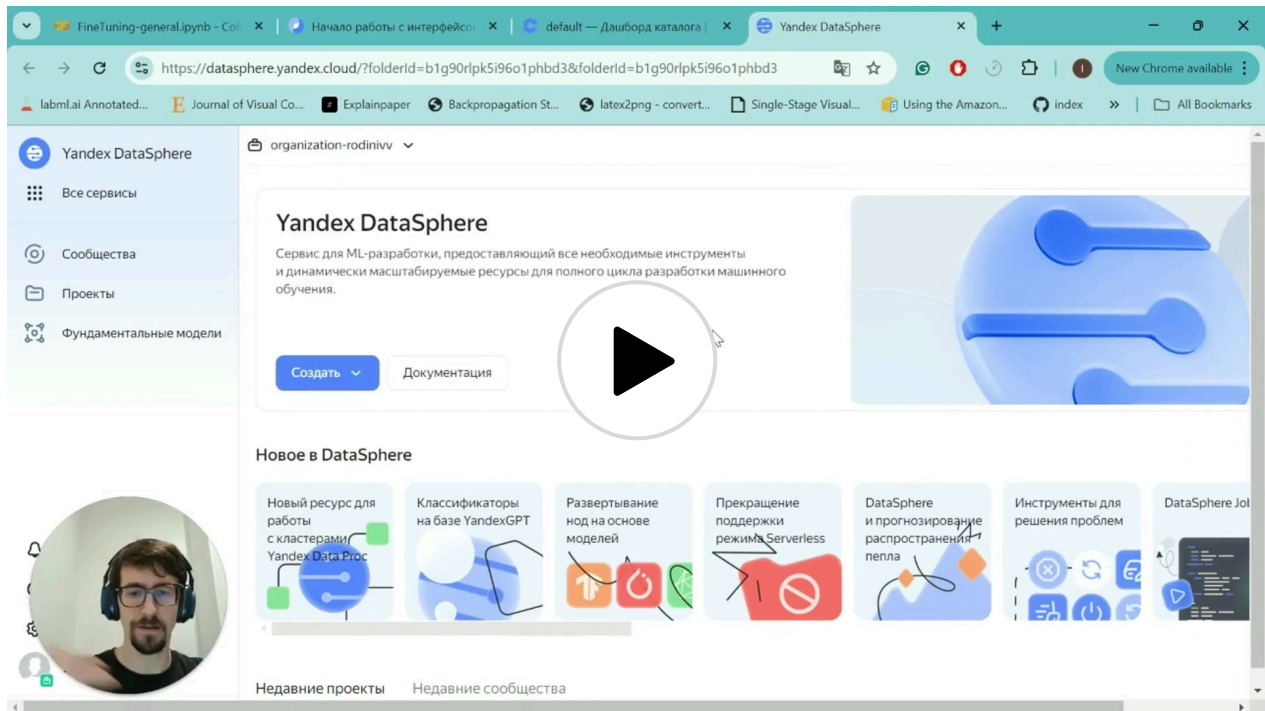
Все сервисы

 Compute Cloud	 Managed Service for PostgreSQL	 AI Studio >
 Managed Service for MongoDB	 Managed Service for Redis	 Foundation Models
 Managed Service for MySQL	 Managed Service for Kafka	 SpeechKit
 Managed Service for OpenSearch	 Managed Service for Apache Airflow™ NEW	 Search API
 Managed Service for Greenplum	 Managed Service for ClickHouse	 Vision OCR
 Object Storage	 Virtual Private Cloud	 Translate
 Network Load Balancer	 Application Load Balancer	 DataSphere
 Container Registry	 Yandex Data Proc	
 Managed Service for Kubernetes	 Managed Service for YDB Yandex Open Source ⓘ	
 Message Queue	 Cloud Functions	
 Serverless Containers	 Serverless Integrations	
 Identity and Access Management	 Key Management Service	
 IoT Core	 Lockbox	
 Certificate Manager	 Data Transfer	
 API Gateway	 Cloud DNS	
 Audit Trails	 Cloud CDN	
 Data Streams Yandex Open Source ⓘ	 Cloud Logging	
 Cloud Desktop PREVIEW	 Yandex SmartCaptcha	
 Managed Service for GitLab	 Load Testing NEW	
 Cloud Apps PREVIEW	 Cloud Postbox NEW	
 Cloud Backup	 WebSQL	
 Smart Web Security	 DataLens Yandex Open Source ⓘ	
 Monitoring	 Yandex Query Yandex Open Source ⓘ	
 Yandex MetaData Hub PREVIEW	 Cloud Notification Service PREVIEW	
 BareMetal PREVIEW	 Code Assistant PREVIEW	
 Marketplace	 Cloud Registry PREVIEW	

Одним из интересных сервисов является **DataSphere**, который служит средой для разработки на Python. Этот инструмент полезен для Fine-Tuning моделей в облачной среде, где можно, например, обучать большие языковые модели.

Также нам интересны хранилище Яндекса и сервис **Compute Cloud** — предоставление виртуальных машин по запросу.

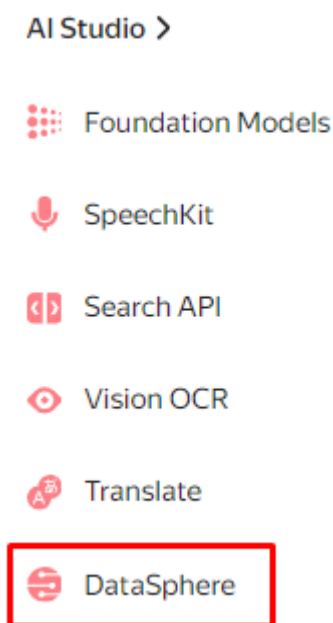
2. Среда разработки DataSphere



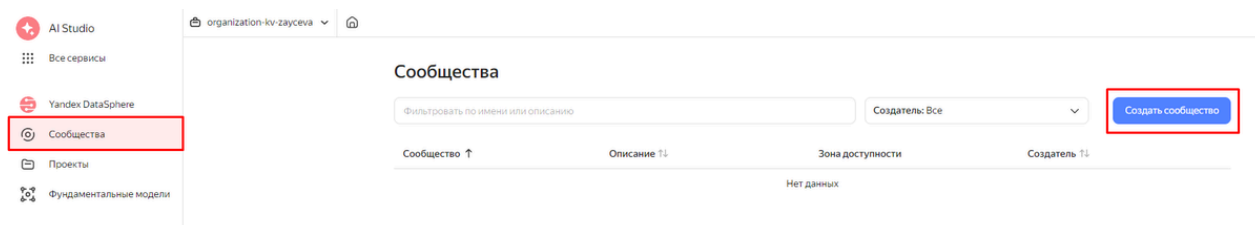
DataSphere похож на Google Colab по интерфейсу и принципу работы: здесь можно создавать проекты, запускать код, сохранять результаты и проводить вычисления.

Допустим, мы хотим разработать Telegram-бот, который будет распознавать жесты (например, «камень, ножницы, бумага») на фотографиях.

1. Нажимаем на **DataSphere**.



2. Выбираем **Сообщество**, и создаем новое. Например, Zerocoder.



3. Вводим данные и нажимаем **Создать**.

Создать сообщество



Имя ?

ZeroCoder

+ Добавить описание

Платежный аккаунт Yandex Cloud

account-934

Зона доступности ?

ru-central1-b

Отмена

Создать

4. Возвращаемся на главную страницу AI Studio. Спускаемся ниже по странице и выбираем **Создать**.

Недавние проекты

Создать проект

Создайте ваш первый проект для начала работы в DataSphere

Создать

Подробнее

5. В поле **Имя** пишем "rpc-bot".

6. В поле **Сообщество** выбираем "zerocoder". Сообщество это группа людей, которые будут иметь доступ к проекту.

7. Нажимаем на кнопку **Создать**.

8. Открываем созданный проект. Нажимаем на кнопку **Открыть проект в JupyterLab**.

JupyterLab Неактивный Остановить JupyterLab и VM



Нет активных конфигураций
Когда вы запустите проект, здесь появится информация о конфигурациях вычислительных ресурсов проекта.
[Открыть проект в JupyterLab](#)

Открытие проекта в **Jupyter Lab** может занять некоторое время. Будь готов подождать 10–15 минут.

Если проект выдает ошибку, попробуй создать новое сообщество и изменить зону доступности.

Создать сообщество ✕

Имя ?

Zerocoder

+ Добавить описание

Платежный аккаунт Yandex Cloud

account-934

Зона доступности ?

ru-central1-b

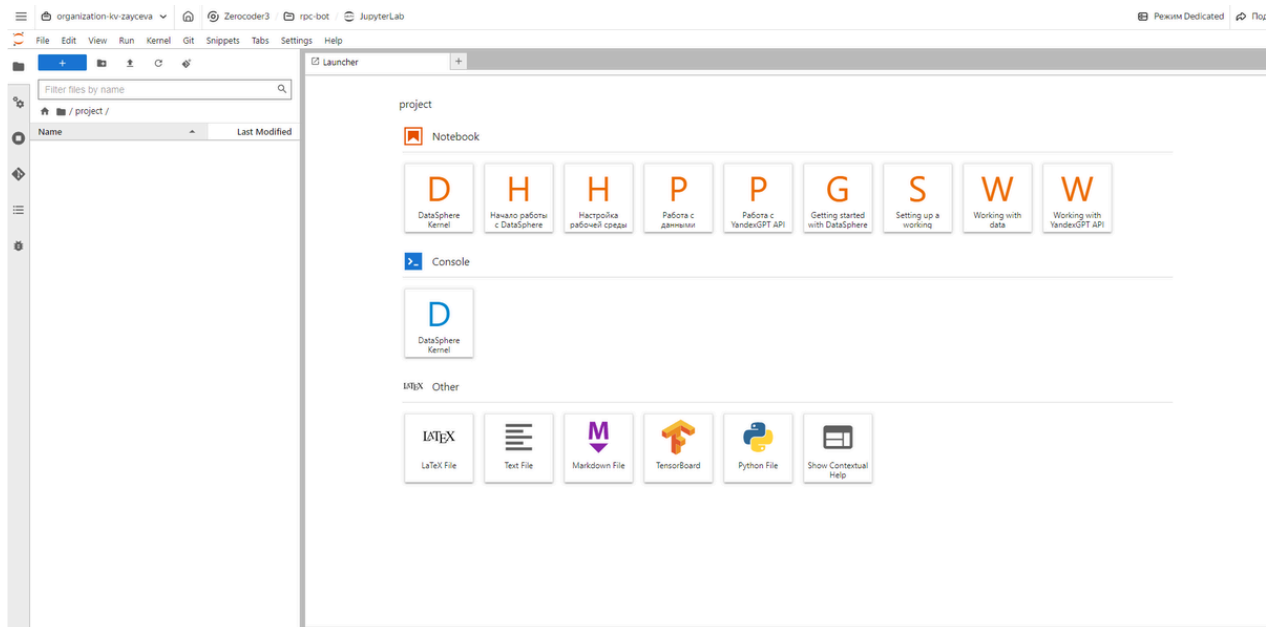
Отмена

Создать

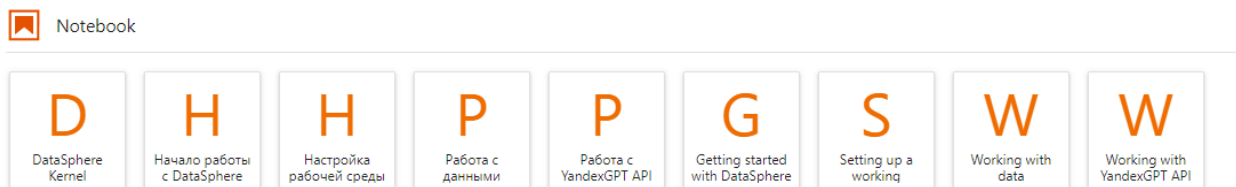
💡 **JupyterLab** — это среда разработки, которая работает по запросу, предоставляя временный доступ к виртуальной машине.

Она напоминает Google Colab: ячейки для кода, файловая система для хранения и доступа к файлам. Основная цель — быстрые вычисления и сохранение результатов, поэтому код работает после запуска в течение ограниченного времени.

Запускаем JupyterLab.



На стартовой странице мы можем найти примеры ноутбуков.



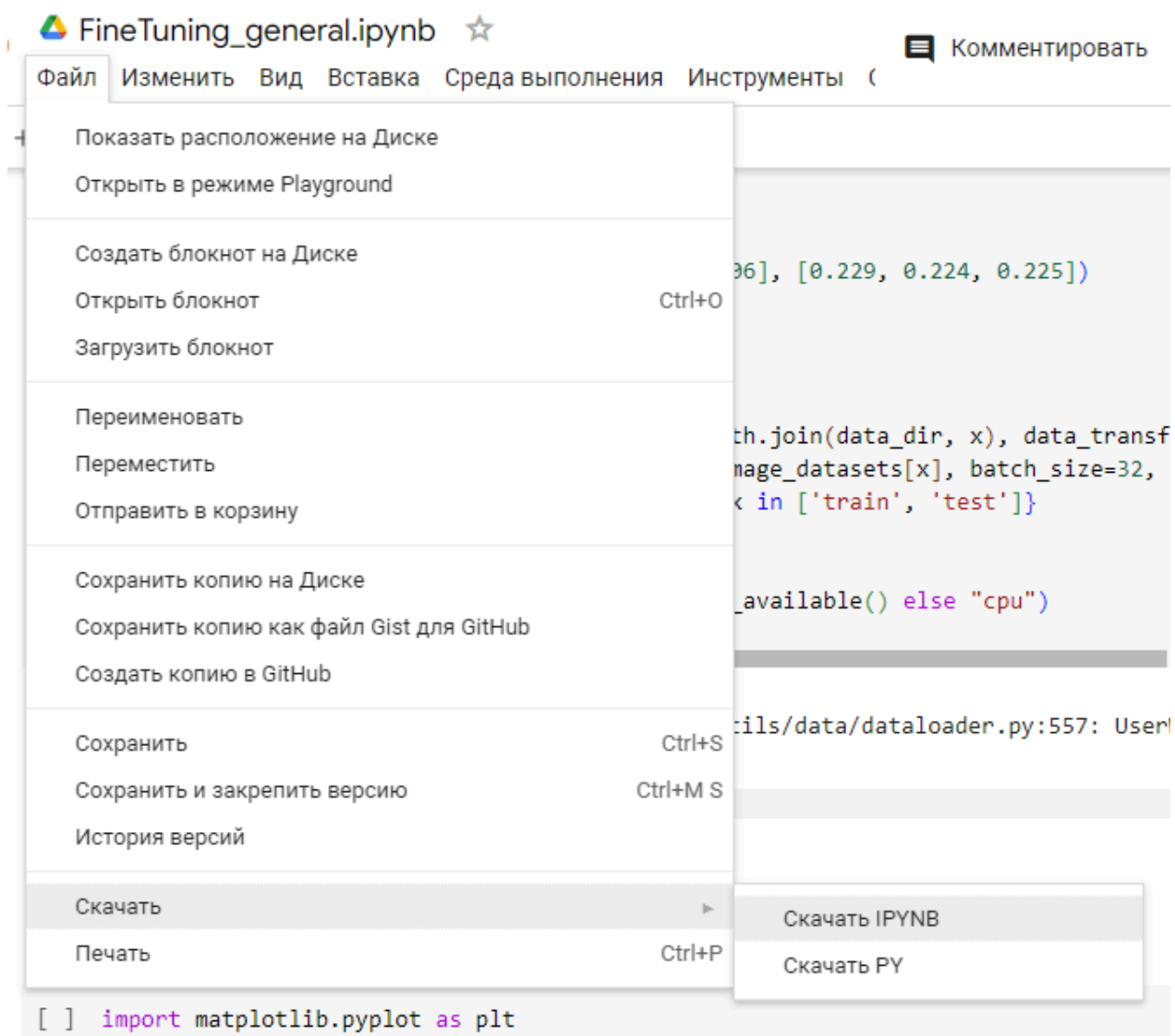
Добавляем свой проект:

1. Переходим в **Google Colab** и открываем ноутбук FineTuning-general.ipynb.

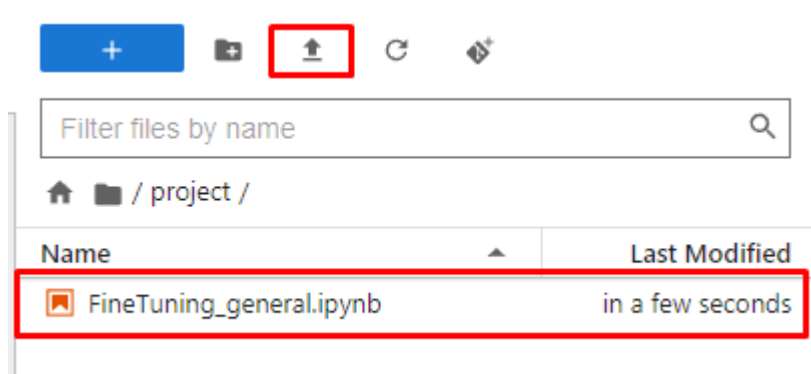
https://colab.research.google.com/drive/1F_5K0m1UFxt8Wrlf1BLPqlV1fMmuLkOj?usp=sharing

(https://colab.research.google.com/drive/1F_5K0m1UFxt8Wrlf1BLPqlV1fMmuLkOj?usp=sharing)

2. Выбираем **File - Download - Download .ipynb**.



3. Переходим в **JupyterLab**. С помощью кнопки в левой панели подгружаем скачанный файл.



4. Адаптируем код под новый формат (заменяем служебные команды):

- вместо `!pip` делаем `%pip`;
- удаляем `>> None`.

Исправленный код:

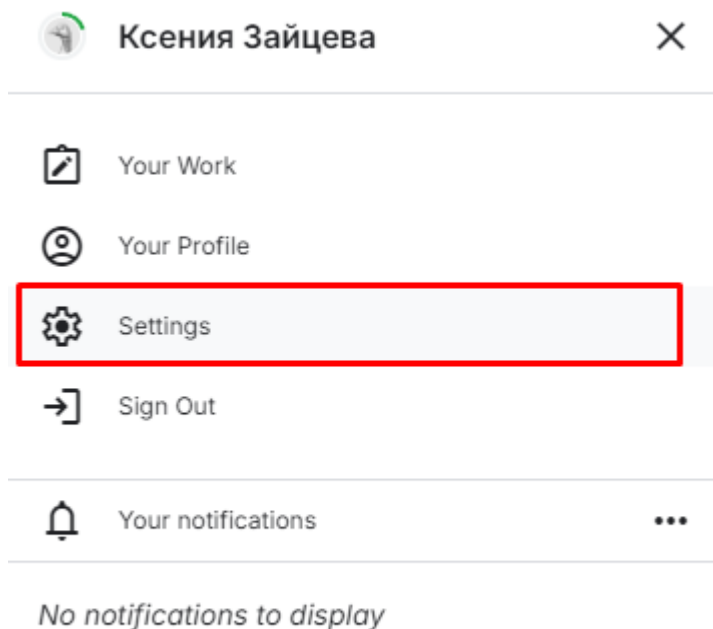
```
# Install Kaggle package
%pip install kaggle

# Download the Rock Paper Scissors dataset from Kaggle
!kaggle datasets download -d sanikamal/rock-paper-scissors-dataset

# Unzip the dataset
!unzip rock-paper-scissors-dataset.zip
```

5. Загружаем файл **kaggle.json**. Этот файл позволяет логиниться в аккаунт и скачивать датасеты. Этот файл скачивается в настройках аккаунта на kaggle.com (<http://kaggle.com/>):

- Переходим на сайт и регистрируем аккаунт.
- Переходим в настройки аккаунта.



- Переходим в раздел API и нажимаем на кнопку **Create new token**.

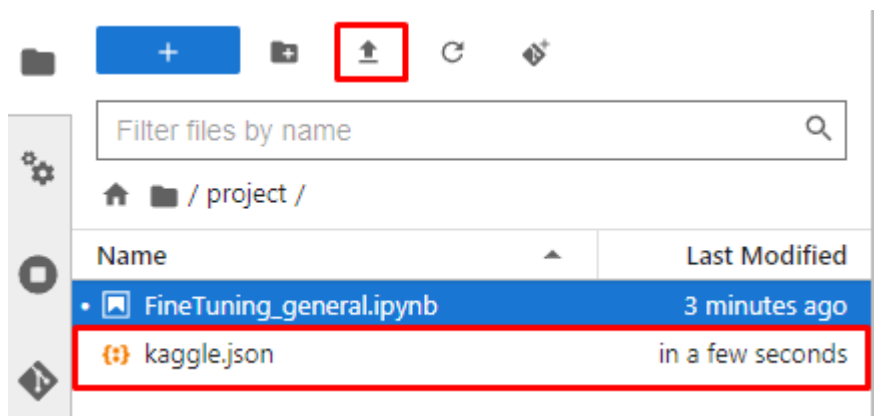
API

Using Kaggle's beta API, you can interact with Competitions and Datasets to download data, make submissions, and more via the command line. [Read the docs](#)

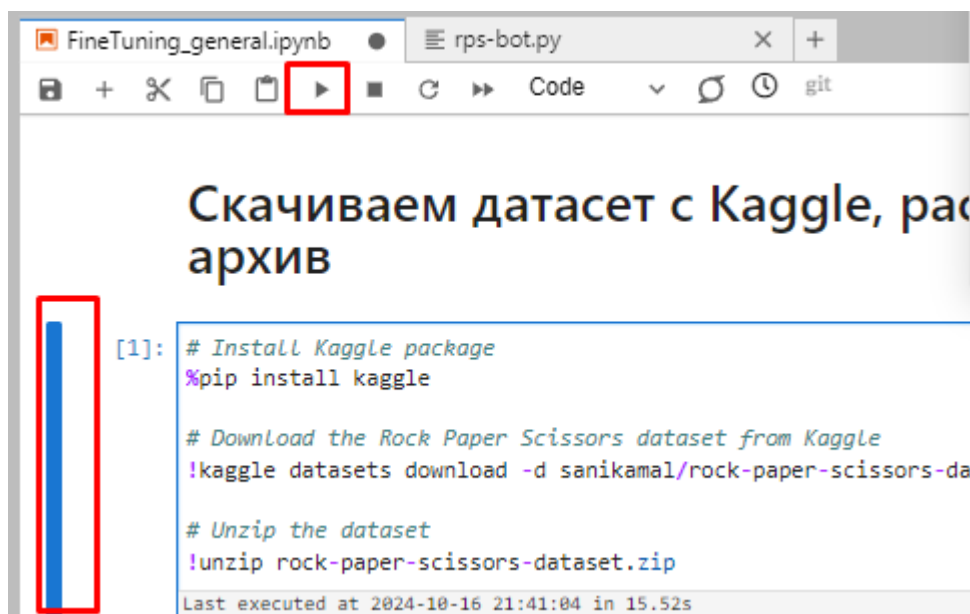
Create New Token

Expire Token

- Автоматически скачивается файл, который необходимо загрузить в наш проект.



6. Запускаем первую ячейку, нажав на треугольник в верхней части, и ждем, пока загрузятся датасеты. Синяя полоска указывает на то, что ячейка выбрана.



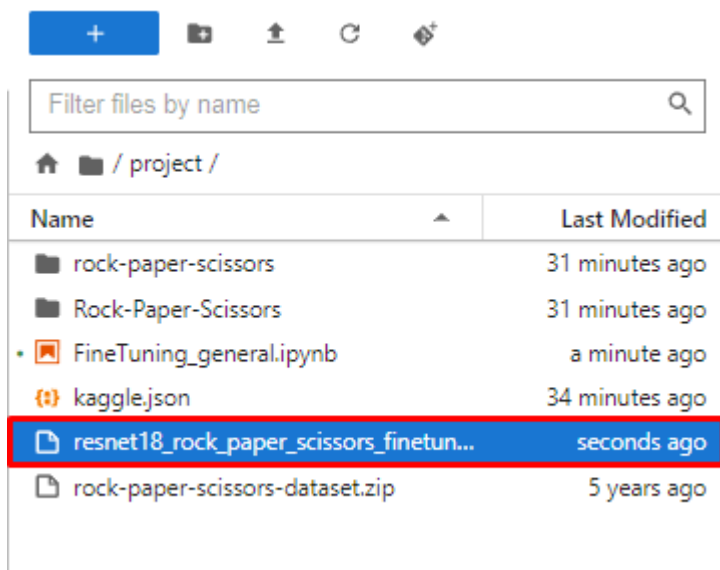
7. По очереди запускаем остальные ячейки до текста **"Вывод"**.

8. Ждём, пока завершатся все вычисления (этот процесс может занять более 30 минут).

```
Training ResNet-18 from scratch
Epoch 0/4
-----
100% [██████████] 79/79 [03:28<00:00, 2.64s/it]
train Loss: 1.0133 Acc: 0.4770
100% [██████████] 12/12 [00:10<00:00, 1.10it/s]
test Loss: 1.2869 Acc: 0.4435
Epoch 1/4
-----
100% [██████████] 79/79 [03:32<00:00, 2.69s/it]
train Loss: 0.7205 Acc: 0.7135
100% [██████████] 12/12 [00:10<00:00, 1.11it/s]
test Loss: 2.8276 Acc: 0.5484
Epoch 2/4
-----
100% [██████████] 79/79 [03:27<00:00, 2.63s/it]
train Loss: 0.5328 Acc: 0.8067
100% [██████████] 12/12 [00:10<00:00, 1.12it/s]
test Loss: 2.4364 Acc: 0.5672
Epoch 3/4
-----
100% [██████████] 79/79 [03:27<00:00, 2.63s/it]
train Loss: 0.4221 Acc: 0.8444
100% [██████████] 12/12 [00:11<00:00, 1.09it/s]
test Loss: 1.7347 Acc: 0.7419
Epoch 4/4
-----
100% [██████████] 79/79 [03:28<00:00, 2.64s/it]
train Loss: 0.3782 Acc: 0.8548
100% [██████████] 12/12 [00:10<00:00, 1.12it/s]
test Loss: 1.6898 Acc: 0.6317
Training complete in 18m 20s
Best val Acc: 0.7419
Fine-tuning pretrained ResNet-18
Epoch 0/4
-----
100% [██████████] 79/79 [01:17<00:00, 1.02it/s]
train Loss: 0.8041 Acc: 0.6472
100% [██████████] 12/12 [00:10<00:00, 1.11it/s]
test Loss: 0.5989 Acc: 0.7742
Epoch 1/4
-----
100% [██████████] 79/79 [01:17<00:00, 1.02it/s]
train Loss: 0.4601 Acc: 0.8520
100% [██████████] 12/12 [00:10<00:00, 1.11it/s]
test Loss: 0.4026 Acc: 0.8683
Epoch 2/4
-----
100% [██████████] 79/79 [01:17<00:00, 1.02it/s]
train Loss: 0.3927 Acc: 0.8694
100% [██████████] 12/12 [00:10<00:00, 1.10it/s]
test Loss: 0.3753 Acc: 0.8522
Epoch 3/4
-----
100% [██████████] 79/79 [01:17<00:00, 1.02it/s]
train Loss: 0.3456 Acc: 0.8802
100% [██████████] 12/12 [00:10<00:00, 1.11it/s]
test Loss: 0.3160 Acc: 0.9274
Epoch 4/4
-----
100% [██████████] 79/79 [01:18<00:00, 1.01it/s]
train Loss: 0.3254 Acc: 0.8853
100% [██████████] 12/12 [00:10<00:00, 1.10it/s]
test Loss: 0.3102 Acc: 0.9005
Training complete in 7m 23s
Best val Acc: 0.9274
```

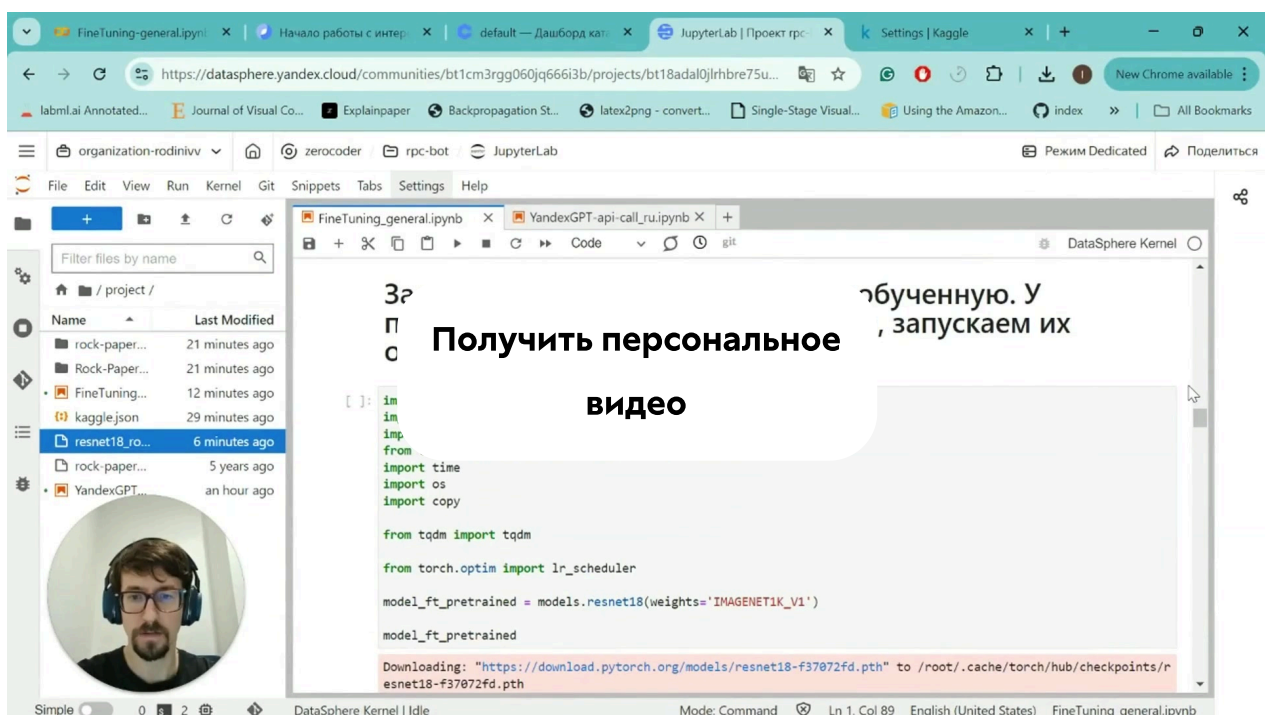
9. Запускаем ячейку для сохранения весов модели:

```
torch.save(model_ft_pretrained.state_dict(), 'resnet18_rock_paper_sc
```



Полученный файл мы можем скачать прямо из структуры слева. Но нам нужно сделать веса доступными для всех приложений Яндекс.Облака.

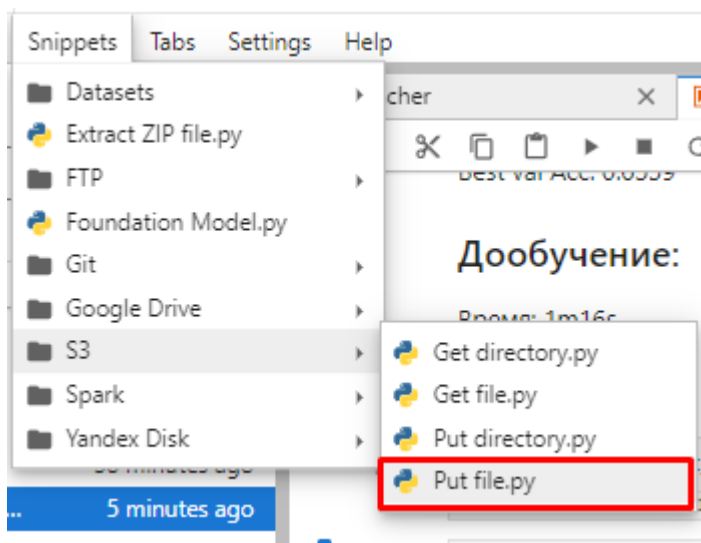
3. Загрузка файлов в Yandex Object Storage



Давайте попробуем полученные веса сохранить на хранилище Яндекс.Облака . Когда мы положим веса в хранилище, они станут доступны для всех других приложений Яндекс.Облака.

1. Сверху поля для кода выбираем вкладку **Snippets**. Здесь находятся фрагменты кода, которые часто вызываются.

2. Пролистываем до S3 → Put file.py . Этот код позволяет отправить файл в хранилище.



В появившемся коде мы видим данные, которых у нас на данный момент нет:

```

# %pip install boto3 if needed
# set os environment variables aws_access_key_id and aws_secret_acce

import os
import boto3

def upload_file(src_file: str, bucket: str, dist_directory: str) ->
    client.upload_file(src_file, bucket, os.path.join(dist_directory

S3_CREDS = {
    "aws_access_key_id": os.environ['aws_access_key_id'],
    "aws_secret_access_key": os.environ['aws_secret_access_key']
}

bucket = "my_bucket"
dist_directory = ""
src_file = 'path/to/file'

client = boto3.client(
    service_name='s3',
    endpoint_url='https://storage.yandexcloud.net',
    **S3_CREDS)

upload_file(src_file, bucket, dist_directory)

```

Для того, чтобы получить данные, совершаем следующие действия:

1. Переходим в дашборд проекта.
2. Выбираем **сервис Object Storage**.



3. Нажимаем на кнопку **Создать бакет**.

4. Вводим имя бакета. Важно выбрать уникальное имя для бакета, так как оно будет использоваться для обращения к этому хранилищу. Обращение может осуществляться по уникальной ссылке. Это необходимо, чтобы система могла однозначно идентифицировать бакет.

5. Выбираем максимальный размер и выбираем другие настройки.

6. Нажимаем на кнопку **Создать бакет**.

Новый бакет

Имя* ?	my-bucket-zerocoder		
Макс. размер	10	ГБ v	☐ Без ограничения
Доступ на чтение объектов ?	<input checked="" type="button" value="Ограниченный"/> Публичный		
Доступ к списку объектов ?	<input checked="" type="button" value="Ограниченный"/> Публичный		
Доступ на чтение настроек ?	<input checked="" type="button" value="Ограниченный"/> Публичный		
Класс хранилища ?	<input checked="" type="button" value="Стандартное"/> Холодное Ледяное		
Метки	Добавить метку		
Создать бакет Отменить			

7. Кликаем на созданный бакет.

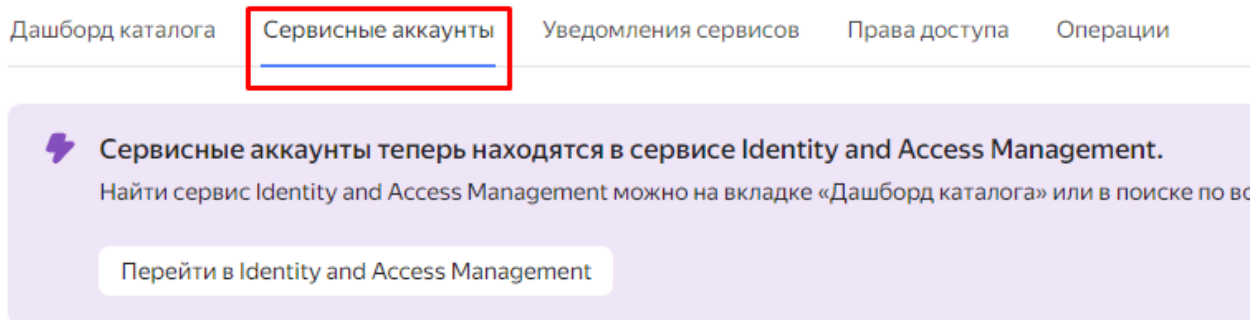
Имя		Занято	Кол-во объектов	⚙
my-bucket-zerocoder	Публичный доступ	0 Б / 10 ГБ		...

8. Чтобы загрузить в него файлы, мы можем нажать на кнопку **Загрузить объекты**, но мы загрузим файл из среды JupyterLab. Это позволит другим сервисам Яндекса иметь доступ к файлу, размещённому в бакете. Для этого внутри кода в поле `bucket = "`указываем имя созданного бакета".

```
bucket = "my-bucket-zerocoder"
dist_directory = ""
src_file = 'resnet18_rock_paper_scissors_finetuned.pth'
```

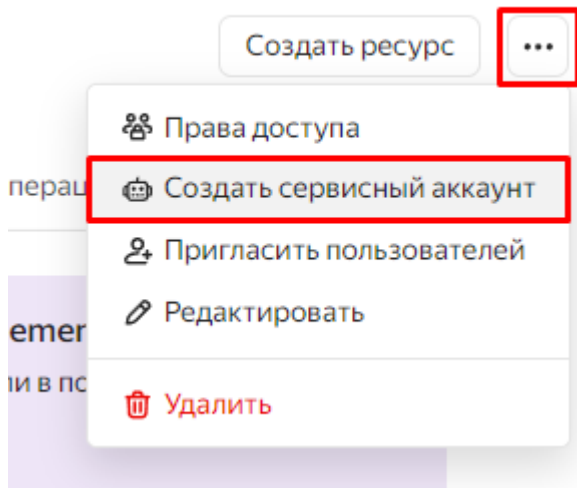
9. Внутри кода в поле `src_file =` вставляем `'resnet18_rock_paper_scissors_finetuned.pth'` .

10. Для того чтобы загрузить файл из JupyterLab, нам необходимо указать **access_key** и **secret_access_key**. Эти ключи позволяют получить доступ к хранилищу. Чтобы получить ключи, возвращаемся на стартовую страницу и переходим во вкладку «Сервисные аккаунты».

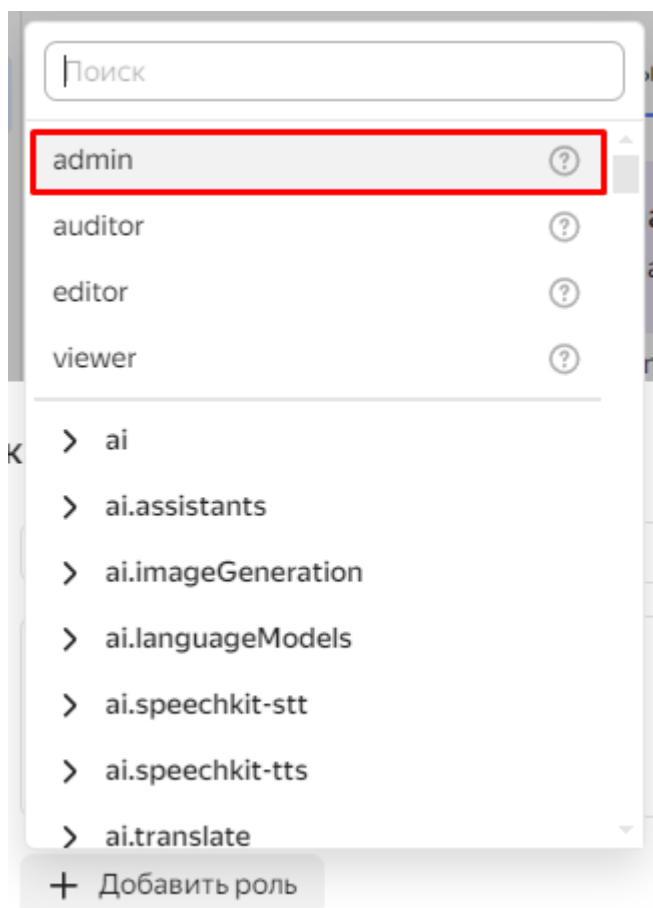


💡 Обрати внимание, что сервисные аккаунты теперь находятся в сервисе Identity and Access Management (<https://console.yandex.cloud/folders/b1g3pf35gk16nug3mkb7/iam/service-accounts>).

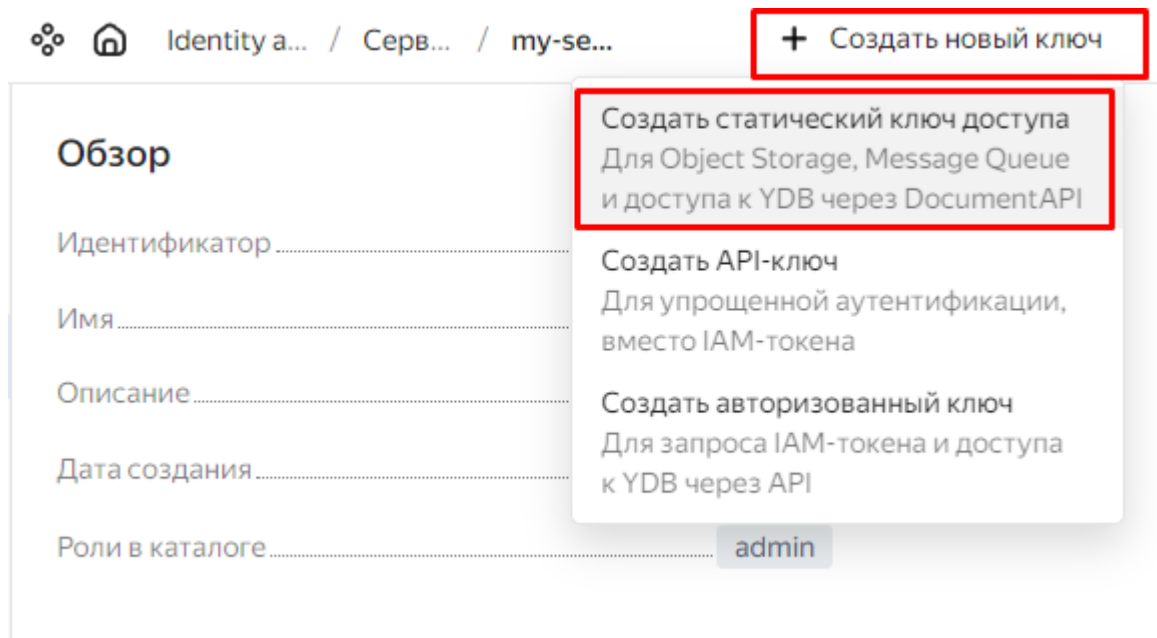
11. Если у тебя ещё нет сервисного аккаунта, создай его. Кликаем на три точки в правом верхнем углу экрана и выбираем **Создать сервисный аккаунт**.



12. Присваиваем аккаунту имя, например, **"my-service-acc-zerocoder"**, и добавляем описание. Нажимаем **Добавить роль** и назначаем ему роль администратора, чтобы предоставить все необходимые права. Затем нажимаем кнопку **Создать**.



13. Когда сервисный аккаунт создан, можно сгенерировать статический ключ. Нажимаем на **Создать новый ключ - Создать статический ключ доступа**. Добавляем описание ключа, нажимаем на кнопку **Создать**.



💡 Система выдаёт идентификатор ключа и секретный ключ. Эти данные **важно хранить в секрете**, так как, обладая ими, кто-то может получить доступ к нашим ресурсам.

14. Теперь, когда ключи получены, копируем их в соответствующие поля (`aws_access_key_id` и `aws_secret_access_key`) и запускаем ячейку в JupyterLab. После успешного выполнения кода файл будет загружен в наш бакет.

```
S3_CREDS = {  
    "aws_access_key_id": "YCAJEx5KxgOBIVBiCat5UdVro",  
    "aws_secret_access_key": "YCOslWC6ngr0x9cemUzfLtjmn6Nm0u0EAK_sPYfxg" I  
}
```

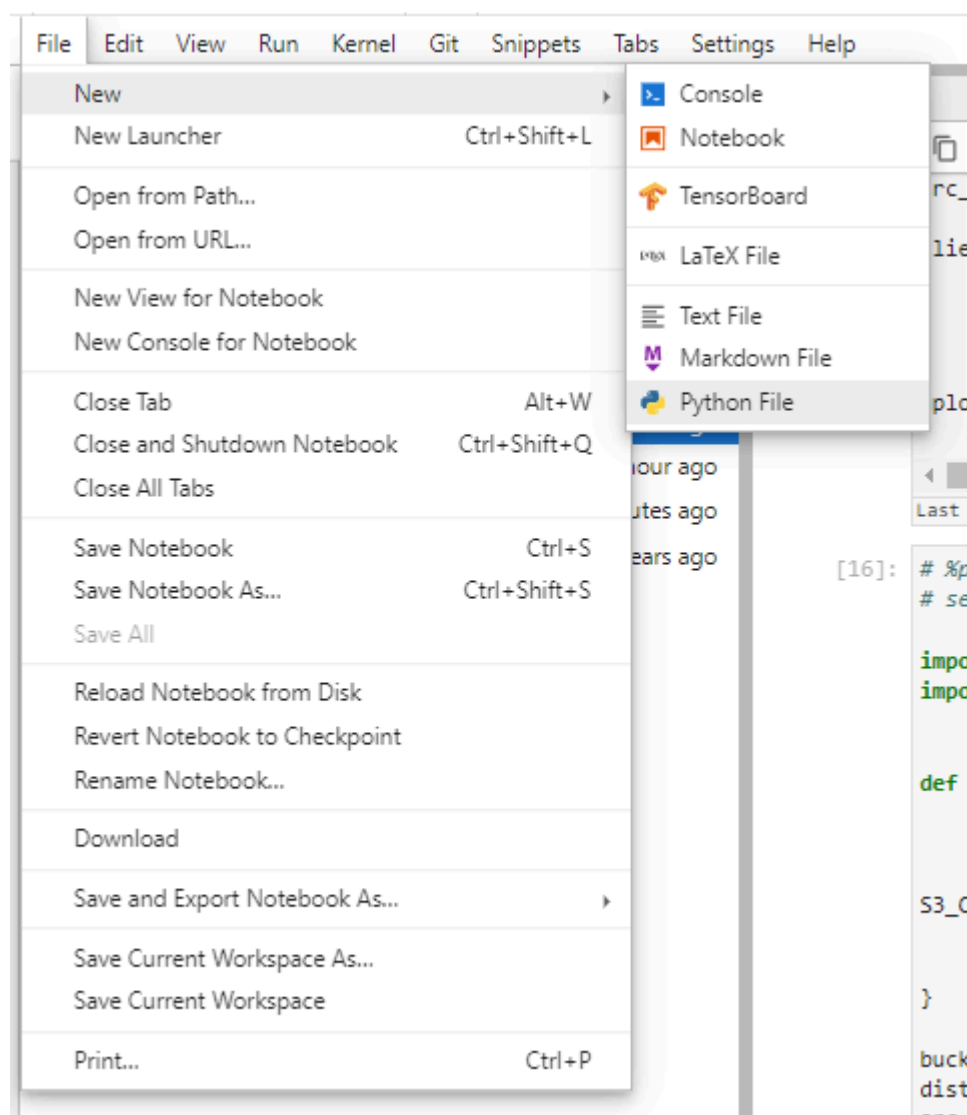
15. Чтобы убедиться, что файл успешно загрузился, возвращаемся на домашнюю страницу и проверяем содержимое нашего **Object Storage**. Если файл в бакете, значит всё прошло успешно.

☐	Имя	Размер	Класс хранилища	Последнее изменение
☐	 resnet18_rock_paper_scissors_finetuned.pth	42.72 МБ	Стандартное	16.10.2024, в 20:09

После загрузки файла мы переходим к созданию файла, который будет запускать простого бота для Telegram.

1. Регистрируем бота в Telegram через BotFather. После регистрации BotFather выдаст нам токен для доступа к боту, который мы указываем в коде.

2. Создаём файл в верхней части структуры JupiterLab: File → New → Python File. Этот файл будет содержать код бота в Telegram.



Содержание файла:

```

import telebot
from telebot.types import Message, PhotoSize
import torch
from torchvision import models, transforms
from PIL import Image
import os

# Your bot token from BotFather
TOKEN = 'ДОБАВЬ СЮДА СВОЙ ТОКЕН' # Replace with your actual bot token

# Create a bot instance
bot = telebot.TeleBot(TOKEN)

# Load your fine-tuned ResNet18 model
model = models.resnet18()
model.fc = torch.nn.Linear(512, 3) # Assuming 3 classes: rock, paper, scissors
model.load_state_dict(torch.load('resnet18_rock_paper_scissors_finetuned.pth'))
model.eval()

# Define transformations to match training preprocessing
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Function to predict the label
def predict_image(image_path):
    image = Image.open(image_path)
    image = transform(image).unsqueeze(0) # Add batch dimension
    with torch.no_grad():
        output = model(image)
        _, predicted = torch.max(output, 1)
    classes = ['paper', 'rock', 'scissors'] # The classes correspond to the indices
    return classes[predicted.item()]

@bot.message_handler(commands=['start'])
def send_welcome(message: Message):
    # Use send_message instead of reply_to
    bot.send_message(message.chat.id, "Send me an image of 'rock', 'paper', or 'scissors'")

# Handle image input

```

```

@bot.message_handler(content_types=['photo'])
def handle_image(message: Message):
    try:
        # Get the highest resolution photo
        file_id = message.photo[-1].file_id
        file_info = bot.get_file(file_id)
        downloaded_file = bot.download_file(file_info.file_path)

        # Save the image
        file_path = "received_image.jpg"
        with open(file_path, 'wb') as new_file:
            new_file.write(downloaded_file)

        # Predict the image using the model
        result = predict_image(file_path)

        # Respond to the user with the classification
        bot.reply_to(message, f"I see: {result}")

        # Clean up the saved image
        os.remove(file_path)
    except Exception as e:
        bot.reply_to(message, f"An error occurred: {e}")

# Start polling to receive messages
bot.polling()

```

3. Используем модель ResNet, которую предварительно обучили для классификации изображений с тремя классами: камень, ножницы или бумага. Мы загружаем веса модели, переводим её в режим предсказаний и прописываем преобразования для входных изображений.

```
# Load your fine-tuned ResNet18 model
model = models.resnet18()
model.fc = torch.nn.Linear(512, 3) # Assuming 3 classes: rock, paper, scissors
model.load_state_dict(torch.load('resnet18_rock_paper_scissors_finetuned.pth'))
model.eval()
```

4. Создаем функцию, которая принимает изображение, применяет к нему необходимые преобразования, передаёт его в модель, и затем возвращает класс с наибольшей вероятностью — камень, ножницы или бумага.

```
# Define transformations to match training preprocessing
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Function to predict the label
def predict_image(image_path):
    image = Image.open(image_path)
    image = transform(image).unsqueeze(0) # Add batch dimension
    with torch.no_grad():
        output = model(image)
        _, predicted = torch.max(output, 1)
    classes = ['paper', 'rock', 'scissors'] # The classes correspond to the indices in the output
    return classes[predicted.item()]
```

5. Пишем обработчики сообщений. Первый обработчик отвечает за приветствие при вводе команды `/start`. Второй — за обработку изображений, отправленных пользователями. После получения фото бот передаёт его в функцию предсказания, и пользователь получает ответ.

```
@bot.message_handler(commands=['start'])
def send_welcome(message: Message):
    # Use send_message instead of reply_to
    bot.send_message(message.chat.id, "Send me an image of 'rock', '

# Handle image input
@bot.message_handler(content_types=['photo'])
def handle_image(message: Message):
    try:
        # Get the highest resolution photo
        file_id = message.photo[-1].file_id
        file_info = bot.get_file(file_id)
        downloaded_file = bot.download_file(file_info.file_path)

        # Save the image
        file_path = "received_image.jpg"
        with open(file_path, 'wb') as new_file:
            new_file.write(downloaded_file)

        # Predict the image using the model
        result = predict_image(file_path)

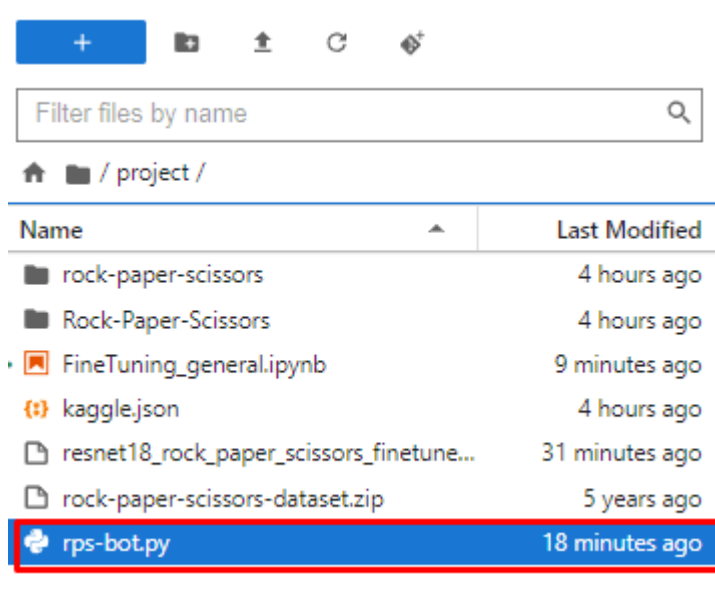
        # Respond to the user with the classification
        bot.reply_to(message, f"I see: {result}")

        # Clean up the saved image
        os.remove(file_path)
    except Exception as e:
        bot.reply_to(message, f"An error occurred: {e}")
```

6. В конце мы запускаем bot polling, чтобы бот начал работать и быть готовым к приёму сообщений.

```
# Start polling to receive messages
bot.polling()
```

7. Загружаем этот файл в свой бакет. Вместо `rps-bot.py` ставим название своего документа, а вместо `my-bucket-zerocoder` название своего бакета:



```
bucket = "my-bucket-zerocoder"
dist_directory = ""
src_file = 'rps-bot.py'

client = boto3.client(
    service_name='s3',
    endpoint_url='https://storage.yandexcloud.net',
    **S3_CREDS)

upload_file(src_file, bucket, dist_directory)
```

4. Настройка виртуальных машин и взаимодействие с ними через командную строку

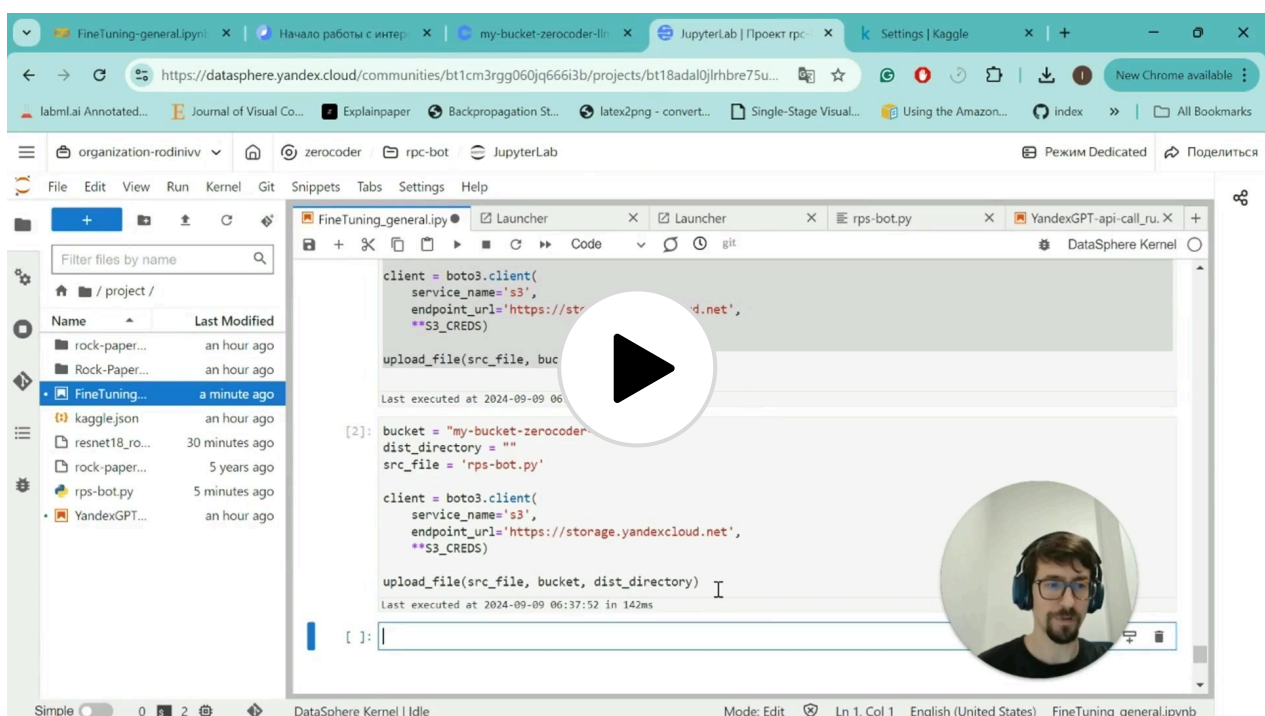
💡 Напоминаем, что важно внимательно отслеживать расходы, чтобы избежать нежелательных списаний с карты:

- **Настраиваем бюджеты и лимиты на расходы.**
- **Отключаем или удаляем неиспользуемые ресурсы.** Особенно важно следить за использованием **виртуальных машин**: обязательно отключаем

их, когда они не используются, так как списания могут происходить даже при отсутствии активности в консоли.

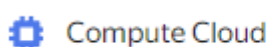
- **Проверяем раздел «Биллинг».**
- Если урок проходит в течение нескольких дней, **убедимся, что все неиспользуемые ресурсы отключены** и на аккаунте не происходит нежелательных списаний.
- **По завершении урока удаляем все созданные проекты и платежные аккаунты** — они будут видны на главной странице.

Просим внимательно отнестись к этим рекомендациям. При возникновении вопросов можно обратиться в поддержку Яндекс.

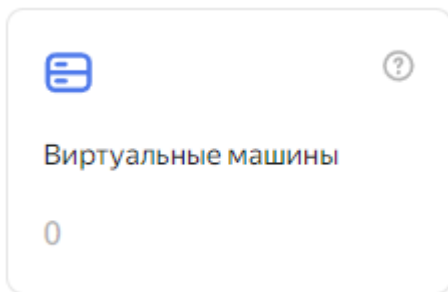


Чтобы бот мог работать постоянно, создаём виртуальную машину в Яндекс.Облаке:

1. Возвращаемся на домашнюю страницу.
2. Выбираем **сервис Compute Cloud**.



3. Кликаем на **Виртуальные машины**.



4. Нажимаем Создать виртуальную машину .

5. Выбираем операционную систему для виртуальной машины, например, Ubuntu. Настраиваем размеры хранилища (по умолчанию 20 ГБ HDD, можно увеличить).

☐ Операционные системы

Ubuntu24.04 ⓘ

CentOS OS Login7 ⓘ

Debian OS Login11 ⓘ

Fedora37 ⓘ

UserGate NGFW6.1.5.11134R ⓘ

Hystax Acura Live Migration to Y... 20... ⓘ

pfSense14.0 ⓘ

Показать все продукты Marketplace

Расположение

Зона доступности ⓘru-central1-b ⌵

Диски и файловые хранилища

ubuntu-24-04-lts-1729100262609Загрузочный ⓘ

Тип ⓘHDDSSDSSD IOНереплицируемый SSD

Размер ⓘ- 20 ГБ + IOPS, bandwidth ⌵
10 ГБ8192 ГБ

Добавить

2 146,40 ₽ в месяц ⌵ ⓘ

Тарифы и цены

Intel Ice Lake. 100% vCPU
1 512,00 ₽

Публичный IP-адрес
172,80 ₽

Intel Ice Lake. RAM
403,20 ₽

Стандартное сетевое хранилище (HDD)
58,40 ₽

6. Определяем необходимую мощность процессоров и видеокарт (если требуется).

Вычислительные ресурсы

Standard High memory HighFreq CPU Shared-core GPU Своя конфигурация

2 vCPU	2 ГБ RAM	2 vCPU	4 ГБ RAM
4 vCPU	8 ГБ RAM	4 vCPU	4 ГБ RAM
8 vCPU	16 ГБ RAM	8 vCPU	8 ГБ RAM

💡 Стоит обратить внимание на цену, которая может сильно увеличиться при использовании мощных видеокарт, таких как Tesla T4.

7. Разрешаем доступ через OS Login

Доступ

Доступ через OS Login ⓘ ☒ Разрешить

8. Нажимаем на кнопку **Создать VM**.

9. После создания виртуальной машины на экране появится команда для подключения к ней через CLI.

Для подключения к машине с помощью CLI используйте команду:
ус compute ssh --name compute-vm-2-2-20-hdd-1729100261872 --folder-i

Подключимся к этой виртуальной машине через командную строку.

Проверь, что статус сменился на Running.

Статус Running

Командная строка, несмотря на её кажущуюся сложность, является довольно простым инструментом, особенно если использовать её для выполнения базовых команд. Эти команды включают, например, просмотр списка файлов, загрузку файлов из интернета или запуск кода. Это возможно в том случае, если код уже протестирован, и нет необходимости выполнять отладку. Командная строка Linux предоставляет широкий набор возможностей, включая установку любых приложений и настройку системы.

Командная строка служит неким "входом" в операционную систему, откуда можно установить приложения, которые затем позволят работать с виртуальной машиной через более удобные интерфейсы, такие как Jupyter Lab, Jupyter Notebook и другие.

Устанавливаем интерфейс командной строки для Яндекс.Облака:

1. Переходим на страницу <https://yandex.cloud/ru/docs/cli/quickstart> (<https://yandex.cloud/ru/docs/cli/quickstart>), где представлен подробный tutorial по установке с помощью командной строки.
2. Переходим на вкладку со своей операционной системой и следуем инструкциям: устанавливаем **интерфейс командной строки Yandex Cloud (CLI)** и **создаем профиль**.

Для Windows CLI можно установить с помощью PowerShell или командной строки:

- Для установки с помощью PowerShell:

1. Выполните команду:

```
iex (New-Object System.Net.WebClient).DownloadString('https://storage.yandexcloud.net/yandexcloud-yc/i
```

2. Скрипт установки спросит, нужно ли добавить путь до `yc` в переменную PATH:

```
Add yc installation dir to your PATH? [Y/n]
```

3. Введите `Y`. После этого CLI Yandex Cloud можно пользоваться, командную оболочку перезапускать не нужно.

- Для установки с помощью командной строки:

1. Выполните команду:

```
@ "%SystemRoot%\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "iex ((New-Object System.Net.W
```

2. Скрипт установки спросит, нужно ли добавить путь до `yc` в переменную PATH:

```
Add yc installation dir to your PATH? [Y/n]
```

3. Введите `Y`.

4. Перезапустите командную оболочку, чтобы перезагрузить переменные окружения.

Если в процессе установки CLI возникла ошибка, см. раздел [Решение проблем CLI](#).

Создание профиля

[Аккаунт на Яндексе](#) [Федеративный аккаунт](#)

Чтобы аутентифицироваться с помощью [аккаунта на Яндексе](#):

1. Получите OAuth-токен в сервисе [Яндекс ID](#):

1.1. Перейдите по [ссылке](#). Если приложение запрашивает доступ к данным, разрешите. Это нужно для получения токена.

1.2. Скопируйте в буфер обмена или сохраните полученный токен.

2. Если вы аутентифицируетесь впервые, перейдите в [консоль облака](#). Примите условия лицензионного соглашения и политики конфиденциальности.

3. Чтобы начать настройку профиля CLI, выполните команду:

```
yc init
```

4. Выберите профиль, для которого вы хотите настроить аутентификацию, или создайте новый. Если вы выполняете команду `yc init` впервые, этот шаг будет отсутствовать.

```
Pick desired action:  
[1] Re-initialize this profile 'default' with new settings  
[2] Create a new profile  
Please enter your numeric choice: 1
```

5. По запросу команды введите OAuth-токен, полученный ранее:

```
Please go to https://oauth.yandex.ru/authorize?response_type=token&client_id=1a6990aa636648e9b2ef855fa7bec:  
in order to obtain OAuth token.
```

```
Please enter OAuth token: y0_AgA ... wvs7N4
```

Как вызвать командную строку:

Пример диалога в консоли:

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.4894]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Ksusha>@"%SystemRoot%\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "iex ((New-Object System.Net.WebClient).DownloadString('https://storage.yandexcloud.net/yandexcloud-yc/install.ps1'))" && SET "PATH=%PATH%;%USERPROFILE%\yandex-cloud\bin"
Downloading yc 0.135.0
Yandex Cloud CLI 0.135.0 windows/amd64

C:\Users\Ksusha>
C:\Users\Ksusha>yc init
Welcome! This command will take you through the configuration process.
Pick desired action:
[1] Re-initialize this profile 'default' with new settings
[2] Create a new profile
[3] Switch to and re-initialize existing profile: 'zerocoder11'
[4] Switch to and re-initialize existing profile: 'zerocoder654'
Please enter your numeric choice: 1
Please go to https://oauth.yandex.ru/authorize?response_type=token&client_id=1a6990aa636648e9b2ef855fa7bec2fb in order to obtain OAuth token.
Please enter OAuth token: [7990224586:*****TnI9lI1iatI] y0_AgAAAAASXDj5AATuwQAAA
AEUxo71AACBY6eY6ULdHLMfEIORAXcza3LoA
Please select cloud to use:
[1] cloud-kv-zayceva (id = b1gonbg8s39h3c0968pk)
[2] llm-zerocoder (id = b1ghf3j20qvmstrpclgh)
Please enter your numeric choice: 1
Your current cloud has been set to 'cloud-kv-zayceva' (id = b1gonbg8s39h3c0968pk).
Please choose folder to use:
[1] default (id = b1g3pf35gk16nug3mkb7)
[2] Create a new folder
Please enter your numeric choice: 1
Your current folder has been set to 'default' (id = b1g3pf35gk16nug3mkb7).
Do you want to configure a default Compute zone? [Y/n] y
Which zone do you want to use as a profile default?
[1] ru-central1-a
[2] ru-central1-b
[3] ru-central1-c
[4] ru-central1-d
[5] Don't set default zone
Please enter your numeric choice: 2
Your profile default Compute zone has been set to 'ru-central1-b'.

C:\Users\Ksusha>yc config list
token: y0_AgAAAAASXDj5AATuwQAAA
cloud-id: b1gonbg8s39h3c0968pk
folder-id: b1g3pf35gk16nug3mkb7
compute-default-zone: ru-central1-b

C:\Users\Ksusha>
```

3. Возвращаемся на страницу виртуальной машины.

4. Копируем указанную команду и вставляем её в командную строку.

Для подключения к машине с помощью CLI используйте команду:

```
yc compute ssh --name compute-vm-2-2-20-hdd-1729100261872 --folder-id b1g3pf35gk16nug3mkb7
```



5. После запуска этой команды мы оказываемся внутри виртуальной машины. В командной строке сначала отображается имя аккаунта, затем знак "@" и имя созданной виртуальной машины.

```
C:\Users\Ksusha>yc compute ssh --name compute-vm-2-2-20-hdd-1729100261872 --folder-id b1g3pf35gk16nu
g3mkb7
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-47-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Oct 16 08:36:54 PM UTC 2024

System load:  0.0               Processes:            132
Usage of /:   54.8% of 19.59GB  Users logged in:     0
Memory usage: 9%               IPv4 address for eth0: 10.129.0.27
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

14 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Wed Oct 16 18:01:25 2024 from 185.147.127.226
kvzayceva@compute-vm-2-2-20-hdd-1729100261872:~$
```

6. Для просмотра файлов в домашней директории виртуальной машины используется команда `ls`. Если ввести `ls /`, можно просмотреть файлы в корневой директории системы. Это позволяет убедиться, что мы находимся на нужной виртуальной машине.

```
kvzayceva@compute-vm-2-2-20-hdd-1729100261872:~$ ls /
Command 'ls' not found, but can be installed with:
apt install ironseed
Please ask your administrator.
kvzayceva@compute-vm-2-2-20-hdd-1729100261872:~$
```

Теперь наша цель — загрузить файлы, которые были ранее сохранены в хранилище.

1. Возвращаемся на главную страницу хранилища.
2. Переходим в Object Storage.

Платежный аккаунт
account-934

Баланс

0,00 ₽

Статус

Active

Потребление ⓘ

3,04 ₽

Готовые решения
Marketplace

125+ продуктов

Инструменты для разработки и анализа данных, обеспечения безопасности, создания бизнес-приложений

Поиск по сервисам

Создать ресурс

Сервисы каталога

Compute Cloud



1

ВМ

1

Диск

Object Storage



1

Бакет

42.72 МБ

Занятое место

Virtual Private Cloud



1

Сеть

2

Подсети

2

Другое

Cloud DNS



2

Зоны

7

Записей

Все сервисы

3. Заходим внутрь файла. Копируем ссылку на файл. Так делаем для двух наших файлов.

Object Storage / Бакеты / my-bucket-zeroCoder / resnet18_rock_paper_scissors_finetuned.pth

Удалить

Получить ссылку

Скачать

...

Последнее изменение 16.10.2024, в 22:43

Размер 42.72 МБ

Класс хранилища Стандартное

Доступ Публичный

4. С помощью команды, в которую мы добавляем скопированную ссылку, скачиваем файл на виртуальную машину при помощи команды:

```
wget ссылка
```

5. После нажатия **Enter** файл будет загружен.

```
kvzayceva@compute-vm-2-2-20-hdd-1729100261872:~$ wget https://storage.yandexcloud.net/my-bucket-zero
coder/resnet18_rock_paper_scissors_finetuned.pth
--2024-10-16 20:41:20-- https://storage.yandexcloud.net/my-bucket-zero/coder/resnet18_rock_paper_sc
issors_finetuned.pth
Resolving storage.yandexcloud.net (storage.yandexcloud.net)... 213.180.193.243, 2a02:6b8::1d9
Connecting to storage.yandexcloud.net (storage.yandexcloud.net)|213.180.193.243|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 44792849 (43M) [application/octet-stream]
Saving to: 'resnet18_rock_paper_scissors_finetuned.pth.1'

resnet18_rock_paper 100%[=====>] 42.72M 66.7MB/s in 0.6s

2024-10-16 20:41:21 (66.7 MB/s) - 'resnet18_rock_paper_scissors_finetuned.pth.1' saved [44792849/447
92849]

kvzayceva@compute-vm-2-2-20-hdd-1729100261872:~$
```

6. Для проверки того, что файл скачался, используем команду `ls`. В списке файлов появится загруженный файл. Аналогичным образом скачивается второй файл. Таким образом, в домашней директории виртуальной машины находятся оба файла — с описанием модели и с кодом для Telegram-бота.

Теперь мы можем запустить код на виртуальной машине.

1. Используем команду:

```
python3 rps-bot.py
```

2. Возникает ошибка, связанная с отсутствием библиотеки **telebot**, которая необходима для работы с ботами в Telegram. Эта ошибка указывает на необходимость установки недостающих модулей.

3. Для установки модулей Python используется команда **pip3 install**. В случае ошибки, например, если PIP3 не установлен, виртуальная машина подскажет команду для установки PIP3 — **apt install python3-pip**. В нашем случае виртуальная машина не содержит предустановленных библиотек, и каждый модуль нужно добавлять вручную.

```
apt install python3-pip
pip3 install telebot
```

Как установить библиотеки на виртуальную машину:

1. Для начала создаем виртуальную среду командой:

```
python3 -m venv myenv
```

2. Активируем созданную среду командой:

```
source myenv/bin/activate
```

После активации в командной строке будет отображаться имя среды, например:

```
(myenv) kvzayceva@compute-vm-2-2-20-hdd-1729100261872:~$
```

3. Теперь, когда виртуальная среда активирована, установим все нужные библиотеки:

```
pip install pyTelegramBotAPI torch torchvision Pillow
```

Эти библиотеки необходимы для работы с Telegram-ботом, PyTorch, моделями и изображениями.

4. Теперь можно запустить ваш код:

```
python rps-bot.py
```

Для проверки работы бота:

1. Переходим в Telegram-бота.
2. Отправляем команду **/start** в Telegram. Если бот прислал приветственное сообщение, это означает, что всё работает корректно.
3. Отправляем боту изображения. К примеру, мы можем использовать изображения из дата-сета, загруженного с Kaggle. Бот должен классифицировать их по принципу "камень", "ножницы", "бумага".

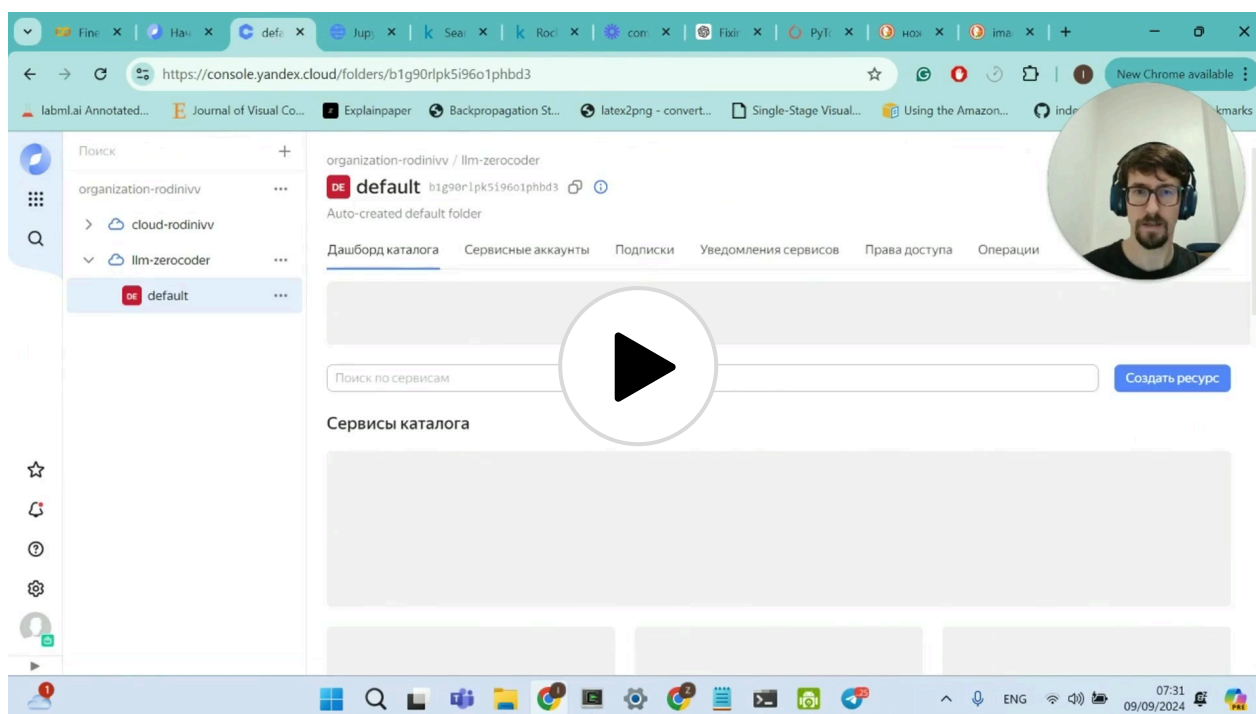






Важно помнить, что качество работы модели зависит от многих факторов, и в реальных условиях нейросеть может давать ошибки.

Основные выводы урока



Мы рассмотрели:

- как работать с Yandex.Cloud, какие сервисы доступны, как создать аккаунт, как привязывать карту (важно следить за потребляемыми ресурсами);
- плюсы и минусы среды разработки DataSphere: ресурсы выдаются на ограниченное время, не предоставляется видеокарта. Среда подходит для запуска простого кода, для визуализации данных, для простых манипуляций с файлами;
- как работает Yandex Storage, как загружать файлы вручную и с помощью кода;
- как настраивать виртуальные машины и взаимодействовать с ними через командную строку, из чего складывается стоимость;
- документацию и интерфейс, их недостатки и преимущества.

Дополнительные материалы

Код, который эксперт использует в примере

<https://replit.com/@zerocodemethodi/Yandex-Cloude#rps-bot.py>
(<https://replit.com/@zerocodemethodi/Yandex-Cloude#rps-bot.py>)

FineTuning_general.ipynb
(<https://fs24.getcourse.ru/fileservice/file/download/a/256825/sc/279/h/21bf8b0fa25d22d8fa968cea6c65d100.ipynb>).
b).

Домашнее задание

Основное задание:

1. Зарегистрируйся в Yandex.Cloud, привяжи карту и активируй бесплатный грант на 4000 рублей (действует 60 дней).
2. Настрой платёжный аккаунт и включи уведомления о расходах, чтобы контролировать траты.

✳️ Важно внимательно отслеживать расходы, чтобы избежать нежелательных списаний с карты. Отключай все сервисы, если не планируешь с ними работать, и удаляй проекты, если выполнил задание и не планируешь к ним возвращаться в ближайшее время.

3. Создай проект в Yandex.Cloud Console, затем создай каталог внутри проекта.
4. Создай и настрой Yandex Object Storage. Создай новый бакет. Настрой бакет и загрузи файл `resnet18_rock_paper_scissors_finetuned.pth`, следуя инструкциям из урока.
5. В **DataSphere** создай новый проект.
6. Внутри проекта создай и открой ноутбук (Jupyter Notebook). Создай или загрузи ноутбук для выполнения Fine-Tuning моделей.
7. Убедись, что код выполняется корректно и результаты отображаются.
8. В качестве домашнего задания пришли скриншоты из консоли со всеми активными сервисами и блокнот с запущенным кодом в Jupyter Notebook.

9. Удали ненужные объекты из Object Storage и отключи неиспользуемые сервисы в проекте, чтобы избежать денежных списаний.

Дополнительное задание:

1. Зарегистрируй Telegram-бота через BotFather и получи токен.
2. Создай в JupyterLab Python-файл `rps-bot.py` с кодом бота. Доработай код.
3. Загрузите файл `rps-bot.py` в свой бакет.
4. В консоли Yandex Cloud выбери Compute Cloud и создай виртуальную машину. Настрой виртуальную машину.
5. Подключись к виртуальной машине через командную строку.
6. Запусти код командой `python3 rps-bot.py` и проверь работу бота, отправив ему изображение через Telegram.
7. В качестве домашнего задания отправь скриншоты из консоли всех подключённых сервисов, содержимое Jupyter Notebook и скриншоты работы Telegram-бота.
8. Заверши работу, удалив все созданные ресурсы в Yandex Cloud (проекты, виртуальные машины, бакеты). Проверь раздел «Биллинг», чтобы исключить нежелательные списания, и убедись, что все ресурсы, не используемые на постоянной основе, отключены.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

РЕ9.4 Российское облачное решение Yandex Cloud

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно

☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

Основное задание:

1. Зарегистрируйся в Yandex.Cloud, привяжи карту и активируй бесплатный грант на 4000 рублей (действует 60 дней).
2. Настрой платёжный аккаунт и включи уведомления о расходах, чтобы контролировать траты.
 Важно внимательно отслеживать расходы, чтобы избежать нежелательных списаний с карты. Отключай все сервисы, если не планируешь с ними работать, и удаляй проекты, если выполнил задание и не планируешь к ним возвращаться в ближайшее время.
3. Создай проект в Yandex.Cloud Console, затем создай каталог внутри проекта.
4. Создай и настрой Yandex Object Storage. Создай новый бакет. Настрой бакет и загрузи файл `resnet18_rock_paper_scissors_finetuned.pth`, следуя инструкциям из урока.
5. В **DataSphere** создай новый проект.
6. Внутри проекта создай и открой ноутбук (Jupyter Notebook). Создай или загрузи ноутбук для выполнения Fine-Tuning моделей.

7. Убедись, что код выполняется корректно и результаты отображаются.
8. В качестве домашнего задания пришли скриншоты из консоли со всеми активными сервисами и блокнот с запущенным кодом в Jupyter Notebook.
9. Удали ненужные объекты из Object Storage и отключи неиспользуемые сервисы в проекте, чтобы избежать денежных списаний.

Дополнительное задание:

1. Зарегистрируй Telegram-бота через BotFather и получи токен.
2. Создай в JupyterLab Python-файл `rps-bot.py` с кодом бота. Доработай код.
3. Загрузите файл `rps-bot.py` в свой бакет.
4. В консоли Yandex Cloud выбери Compute Cloud и создай виртуальную машину. Настрой виртуальную машину.
5. Подключись к виртуальной машине через командную строку.
6. Запусти код командой `python3 rps-bot.py` и проверь работу бота, отправив ему изображение через Telegram.
7. В качестве домашнего задания отправь скриншоты из консоли всех подключённых сервисов, содержимое Jupyter Notebook и скриншоты работы Telegram-бота.
8. Заверши работу, удалив все созданные ресурсы в Yandex Cloud (проекты, виртуальные машины, бакеты). Проверь раздел «Биллинг», чтобы исключить нежелательные списания, и убедись, что все ресурсы, не используемые на постоянной основе, отключены.



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Предыдущий
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=338667562>)

Следующий урок